

100 道 Spring 高频面试题及答案

Spring 基础

1. 什么是 Spring 框架？

- 答：Spring 框架是一个开源的轻量级 Java 开发框架，它提供了一种基于依赖注入（Dependency Injection）和面向切面编程（Aspect - Oriented Programming）的方式来构建企业级应用程序，旨在简化企业级应用开发，提高开发效率和代码的可维护性。

2. Spring 有哪些主要模块？

- 答：Spring 主要模块包括 Spring Core（核心容器）、Spring AOP（面向切面编程）、Spring ORM（对象关系映射）、Spring DAO（数据访问对象）、Spring MVC（模型 - 视图 - 控制器）等。

3. 什么是 IOC（控制反转）？

- 答：IOC 是一种设计思想，将对象的创建和依赖关系的管理从应用程序代码中转移到 Spring 容器中。由 Spring 容器负责创建对象、管理对象的生命周期以及维护对象之间的依赖关系，使得代码的耦合度降低，提高了代码的可维护性和可扩展性。

4. 什么是 DI（依赖注入）？

- 答 :DI 是实现 IOC 的一种方式 ,通过在运行时将对象所依赖的其他对象自动注入到该对象中 ,而不是在对象内部手动创建或查找依赖对象。常见的注入方式有构造函数注入、Setter 方法注入和接口注入。

5. Spring 容器有哪些 ?

- 答 : Spring 容器主要有 BeanFactory 和 ApplicationContext。BeanFactory 是基础容器 , 提供了基本的 IOC 功能 ; ApplicationContext 是 BeanFactory 的子接口 , 在其基础上增加了许多企业级功能 , 如支持国际化、资源加载、事件发布等。

Spring Bean

6. Bean 的作用域有哪些 ?

- 答 : Spring 中 Bean 的作用域有 singleton (单例) 、 prototype (原型) 、 request (请求) 、 session (会话) 、 application (应用) 等。singleton 是默认作用域 , 每个 Spring 容器中只有一个单例 Bean 实例 ; prototype 每次请求都会创建一个新的 Bean 实例 ; request 作用域的 Bean 在一次 HTTP 请求中有效 ; session 作用域的 Bean 在一个用户会话期间有效 ; application 作用域的 Bean 在整个 Web 应用程序中有效。

7. 如何在 Spring 中配置 Bean ?

- 答 : 可以通过 XML 配置文件、Java 配置类或注解的方式来配置 Bean。XML 配置文件中使用 `<bean>` 标签来定义 Bean ; Java 配置类使用 `@Configuration` 注解标记类 , 在方法上使用 `@Bean` 注解定义 Bean ; 注解方式则直接在类上使用 `@Component` 等注解 , 还可以配合 `@Autowired` 等注解进行依赖注入。

8. Bean 的生命周期是什么 ?

- 答 : Bean 的生命周期包括实例化、属性赋值、初始化、使用和销毁。Spring 容器首先创建 Bean 实例 , 然后为其注入属性 , 接着调用初始化方法 (如实现 `InitializingBean` 接口的 `afterPropertiesSet` 方法或使用 `@PostConstruct` 注解的方法) 进行初始化 , 之后 Bean 就可以被应用程序使用 , 当 Spring 容器关闭时 , 会调用销毁方法 (如实现 `DisposableBean` 接口的 `destroy` 方法或使用 `@PreDestroy` 注解的方法) 来销毁 Bean。

9. 如何解决 Bean 的循环依赖问题 ?

- 答 : Spring 通过提前暴露 Bean 的早期引用 (`early - singleton - object`) 来解决循环依赖问题。在创建 Bean 的过程中 , 当完成实例化但还未进行属性赋值和初始化时 , 就将 Bean 的引用放入一个早期单例对象池中 , 其他 Bean 如果需要依赖该 Bean , 就可以从这个池中获取到早期引用 , 从而打破循环依赖。不过 , 这种方式只适用于单例 Bean 的构造函数注入和 Setter 方法注入 , 如果

是原型 Bean 或使用了`@Scope("prototype")`注解的 Bean ,Spring 无法解决循环依赖问题，会抛出异常。

10. @Autowired 注解的作用是什么？

- 答：`@Autowired`注解用于自动装配 Bean 之间的依赖关系。它可以应用在构造函数、Setter 方法、字段或方法参数上，Spring 容器会根据类型自动在容器中查找匹配的 Bean 并注入。如果有多个匹配的 Bean，可以结合`@Qualifier`注解指定具体要注入的 Bean。

Spring AOP

11. 什么是 AOP (面向切面编程)？

- 答：AOP 是一种编程范式，它将横切关注点（如日志记录、事务管理、权限控制等）从业务逻辑中分离出来，以切面（Aspect）的形式进行统一管理和处理。通过 AOP，可以在不修改业务逻辑代码的情况下，在方法执行的前后、抛出异常时等不同的连接点（Joinpoint）插入额外的逻辑，提高了代码的可维护性和复用性。

12. AOP 中的关键概念有哪些？

- 答：AOP 中的关键概念包括切面（Aspect）、连接点（Joinpoint）、切点（Pointcut）、通知（Advice）和引入（Introduction）。切面是横切关注点的模

块化，包含通知和切点等；连接点是程序执行过程中的特定点，如方法调用、异常抛出等；切点用于定义哪些连接点会被织入切面的通知；通知是在连接点上执行的具体操作，包括前置通知、后置通知、环绕通知、异常通知和最终通知等；引入允许在不修改现有类代码的情况下，为类添加新的方法或属性。

13. Spring AOP 是如何实现的？

- 答：Spring AOP 默认使用动态代理来实现，主要有 JDK 动态代理和 CGLIB 代理两种方式。如果目标对象实现了接口，Spring 会使用 JDK 动态代理创建代理对象；如果目标对象没有实现接口，Spring 则会使用 CGLIB 代理生成子类来实现 AOP 功能。在运行时，Spring 会根据切点的定义，将切面的通知织入到目标对象的方法调用中，从而实现 AOP 的功能。

14. @Aspect 注解的作用是什么？

- 答：`@Aspect` 注解用于标记一个类为切面类，表明该类包含了与横切关注点相关的通知和切点等定义。被 `@Aspect` 注解的类会被 Spring AOP 框架识别并进行相应的处理，将其切面逻辑织入到目标对象的方法调用中。

15. 如何在 Spring 中配置 AOP？

- 答：可以通过 Java 配置类或 XML 配置文件来配置 AOP。在 Java 配置类中，使用 `@Configuration` 和 `@EnableAspectJAutoProxy` 注解开启 AOP 自动代

理功能，然后定义切面类并使用`@Aspect`注解标记，在切面类中通过`@Pointcut`注解定义切点，通过`@Before`、`@After`、`@Around`等注解定义通知。在 XML 配置文件中，需要配置`<aop:config>`元素来定义 AOP 配置，包括切点、切面和通知等。

Spring MVC

16. 什么是 Spring MVC ?

- 答：Spring MVC 是 Spring 框架中的一个模块，用于构建 Web 应用程序的 MVC 架构。它将 Web 应用程序分为模型（Model）、视图（View）和控制器（Controller）三个部分，通过将请求处理逻辑、业务逻辑和视图展示分离，提高了应用程序的可维护性和可扩展性。

17. Spring MVC 的工作流程是什么？

- 答：用户发送请求到前端控制器`DispatcherServlet`；`DispatcherServlet`根据请求的URL调用`HandlerMapping`找到对应的处理器（Controller方法）；`DispatcherServlet`调用`HandlerAdapter`来执行处理器；处理器执行完成后返回一个`ModelAndView`对象给`DispatcherServlet`；`DispatcherServlet`根据`ViewResolver`解析视图名称找到对应的视图；最后`DispatcherServlet`将模型数据渲染到视图上并返回给用户。

18. @Controller 注解的作用是什么？

- 答：`@Controller` 注解用于标记一个类为 Spring MVC 中的控制器类，该类中的方法用于处理用户的请求。被 `@Controller` 注解的类会被 Spring 容器扫描并注册为 Bean，同时 Spring MVC 会识别其中的请求处理方法，并将用户的请求映射到相应的方法进行处理。

19. @RequestMapping 注解的作用是什么？

- 答：`@RequestMapping` 注解用于将 HTTP 请求映射到控制器类中的方法。它可以在类级别和方法级别上使用，类级别上的 `@RequestMapping` 定义了该控制器类处理的请求路径的公共前缀，方法级别上的 `@RequestMapping` 则进一步细化了具体的请求路径和请求方法等条件，只有符合这些条件的请求才会被映射到对应的方法进行处理。

20. 如何在 Spring MVC 中处理表单数据？

- 答：在 Spring MVC 中，可以通过在控制器方法的参数中使用 `@ModelAttribute` 注解来绑定表单数据到 Java 对象。首先在 JSP 或其他视图页面中创建表单，表单的字段名称要与 Java 对象的属性名称一致。然后在控制器方法中，使用 `@ModelAttribute` 注解标记一个参数，Spring MVC 会自动将表单数据绑定到该对象的对应属性上。也可以使用 `@RequestParam` 注解来获取单个表单参数的值。

Spring 数据访问

21. Spring 中如何进行数据库操作？

- 答：Spring提供了多种方式进行数据库操作，常用的有Spring JDBC、Spring Data JPA 等。Spring JDBC 通过`JdbcTemplate`类来简化 JDBC 操作，提供了一系列方法来执行 SQL 语句、查询结果集等。Spring Data JPA 则是基于 JPA 规范，通过定义接口并继承`JpaRepository`等接口，就可以自动获得一些基本的数据访问方法，如保存、查询、删除等，同时还可以通过自定义方法名来实现特定的查询逻辑。

22. 什么是 JdbcTemplate？

- 答：`JdbcTemplate`是 Spring JDBC 中的核心类，它封装了 JDBC 的操作，提供了统一的 API 来执行各种数据库操作，如执行 SQL 语句、更新数据、查询数据等。使用`JdbcTemplate`可以避免大量的重复代码，提高数据库操作的效率和代码的可读性，同时它还提供了事务管理等功能。

23. Spring Data JPA 中的 Repository 接口有什么作用？

- 答：`Repository`接口是 Spring Data JPA 中的核心接口，它定义了基本的数据访问方法，如`findAll()`、`findById()`、`save()`、`delete()`等。通过让自定义的接口继承`Repository`接口或其扩展接口（如`JpaRepository`），Spring

Data JPA 会自动为这些接口生成实现类 ,从而实现了数据库的基本 CRUD 操作。同时 ,还可以在自定义接口中定义根据方法名自动解析的查询方法 ,方便进行复杂的数据库查询操作。

24. 如何在 Spring 中配置数据源 ?

- 答 :可以通过多种方式在 Spring 中配置数据源。在 XML 配置文件中 ,使用 `<bean>` 标签定义数据源 Bean ,例如使用 `DriverManagerDataSource` 或连接池数据源如 `C3P0DataSource`、`DruidDataSource` 等 ,并设置相关的属性 ,如数据库驱动类、URL、用户名、密码等。在 Java 配置类中 ,使用 `@Configuration` 注解标记配置类 ,使用 `@Bean` 注解定义数据源方法 ,在方法中创建并配置数据源对象。也可以使用 Spring Boot 的自动配置功能 ,通过在 `application.properties` 或 `application.yml` 文件中配置数据源相关的属性 ,Spring Boot 会自动创建数据源 Bean。

25. Spring 中的事务管理是如何实现的 ?

- 答 :Spring 中的事务管理有两种方式 :编程式事务管理和声明式事务管理。编程式事务管理通过在代码中直接使用 `TransactionTemplate` 或 `PlatformTransactionManager` 来手动管理事务的开始、提交和回滚。声明式事务管理则是通过在方法或类上使用 `@Transactional` 注解来声明事务的边界和属性 ,Spring 会在方法执行前后自动进行事务的管理 ,这种方式基于

AOP 实现，将事务管理的逻辑从业务逻辑中分离出来，提高了代码的可读性和可维护性。

Spring 其他

26. Spring Boot 的特点是什么？

- 答：Spring Boot 具有以下特点：快速构建 Spring 应用，简化了配置；自动配置 Spring 及相关技术，减少了大量的样板代码和配置文件；提供了内置的服务器，如 Tomcat、Jetty 等，方便部署和运行；支持各种 Starters，方便引入依赖；具备强大的监控和管理功能，可以通过 Actuator 端点查看应用的运行状态等。

27. 什么是 Spring Cloud？

- 答：Spring Cloud 是一系列框架的集合，它构建在 Spring Boot 之上，用于构建微服务架构的应用程序。提供了服务治理、配置管理、服务调用、熔断器、网关等一系列功能，帮助开发者快速构建分布式、弹性、可扩展的微服务系统。

28. Spring 中如何实现定时任务？

- 答：可以使用 `@Scheduled` 注解来实现定时任务。在方法上添加 `@Scheduled` 注解，并指定定时任务的执行时间表达式，Spring 会按照指定的时间间隔或时间点来执行该方法。也可以通过实现 `SchedulingConfigurer`

接口来自定义定时任务的配置和调度逻辑。

29. 如何在 Spring 中进行国际化处理？

- 答：首先创建国际化资源文件，如`messages.properties`、`messages\_zh\_CN.properties`等，在文件中定义不同语言的消息。然后在 Spring 配置文件中配置`ResourceBundleMessageSource`来加载国际化资源文件。在代码中，可以通过`MessageSource`接口的`getMessage`方法根据当前用户的语言环境获取对应的消息。在 JSP 页面中，可以使用`<spring:message>`标签来显示国际化消息。

30. Spring 中如何实现文件上传和下载？

- 答：对于文件上传，在 Spring MVC 的控制器方法中，使用`@RequestParam("file") MultipartFile file`来接收上传的文件，`MultipartFile`提供了获取文件信息和保存文件的方法。在前端页面，使用`<input type="file">`标签来选择文件并提交表单。对于文件下载，设置响应头的`Content - Disposition`属性为`attachment; filename="文件名"`，然后通过`OutputStream`将文件内容写入响应流中，前端浏览器会根据响应头信息进行文件下载。

Spring 基础

31. Spring 框架的设计目标是什么？

- 答 :Spring 框架的设计目标主要包括简化企业级应用开发 ,降低开发复杂度 ;提高代码的可维护性和可测试性 ;促进代码的复用 ;提供灵活的配置和模块化架构 ;支持多种数据访问技术和事务管理 ;以及与其他企业级框架和技术进行集成。

32. Spring 如何实现松耦合？

- 答 : Spring 通过依赖注入 (DI) 和面向接口编程实现松耦合。依赖注入将对象的依赖关系的管理从对象内部转移到外部容器 , 使得对象之间的依赖关系在运行时动态确定。面向接口编程则使得对象之间通过接口进行交互 , 而不是直接依赖具体的实现类 , 当实现类发生变化时 , 只要接口不变 , 对其他对象的影响就很小。

33. Spring 中的事件机制是怎样的？

- 答 : Spring 的事件机制基于观察者模式。事件发布者 (Publisher) 发布事件 (Event) , 事件监听器 (Listener) 监听特定类型的事件 , 当事件被发布时 , 监听器会接收到事件并执行相应的处理逻辑。Spring 中可以通过继承 `ApplicationEvent` 类自定义事件 , 实现 `ApplicationListener` 接口或使用 `@EventListener` 注解定义监听器。

34. Spring 的核心容器启动过程是怎样的？

- 答：首先创建 `ApplicationContext` 实例，例如 `ClassPathXmlApplicationContext` 或 `AnnotationConfigApplicationContext`。然后容器会加载配置文件或扫描注解，解析其中定义的 Bean 信息。接着容器会实例化所有单例作用域的 Bean，并进行属性注入和初始化操作。最后容器会发布 `ContextRefreshedEvent` 事件，表示容器启动完成。

35. Spring 中如何进行日志管理？

- 答：Spring 本身不提供日志实现，而是支持多种日志框架，如 Log4j、Logback 等。可以通过在项目中引入相应的日志框架依赖，并进行配置。例如，使用 Logback 时，在 `logback.xml` 文件中配置日志的输出级别、输出格式和输出位置等。Spring 会自动检测并使用项目中配置的日志框架进行日志记录。

Spring Bean

36. 如何在 Spring 中获取 Bean？

- 答：可以通过 `ApplicationContext` 的 `getBean` 方法来获取 Bean，该方法有两种重载形式，一种是通过 Bean 的名称获取，另一种是通过 Bean 的类型获取。例如：`applicationContext.getBean("beanName")` 或 `applicationContext.getBean(BeanClass.class)`。也可以在需要使用 Bean

的类中使用`@Autowired`或`@Resource`注解进行自动注入。

37. Spring 中 Bean 的初始化顺序是怎样的？

- 答：首先执行 Bean 的实例化，然后进行属性注入。接着，如果 Bean 实现了`BeanNameAware`接口，会调用`setBeanName`方法；如果实现了`BeanFactoryAware`接口，会调用`setBeanFactory`方法；如果实现了`ApplicationContextAware`接口，会调用`setApplicationContext`方法。之后，会调用使用`@PostConstruct`注解的方法或实现`InitializingBean`接口的`afterPropertiesSet`方法。最后，如果在XML配置文件中定义了`init-method`属性，会调用该方法。

38. 如何自定义 Bean 的创建过程？

- 答：可以通过实现`FactoryBean`接口来自定义 Bean 的创建过程。
`FactoryBean`接口有三个方法：`getObject()`用于返回实际的 Bean 实例；
`getObjectType()`用于返回 Bean 的类型；`isSingleton()`用于指定 Bean 是否为单例。在实现类中实现这些方法，就可以自定义 Bean 的创建逻辑。

39. Spring 中如何处理 Bean 的销毁？

- 答：当 Spring 容器关闭时，会销毁单例作用域的 Bean。如果 Bean 实现了`DisposableBean`接口，会调用`destroy`方法；如果使用了`@PreDestroy`注

解,会调用注解标记的方法;如果在 XML 配置文件中定义了`destroy - method`属性,会调用该方法。

40. 如何在 Spring 中动态注册 Bean ?

- 答:可以通过`BeanDefinitionRegistry`接口来动态注册Bean。例如,在`ApplicationContext`中获取`BeanDefinitionRegistry`实例,然后创建`BeanDefinition`对象,设置Bean的相关信息,最后调用`registerBeanDefinition`方法将`BeanDefinition`注册到容器中。

Spring AOP

41. Spring AOP 的通知类型有哪些,分别在什么时机执行?

- 答: Spring AOP 的通知类型包括:
- 前置通知(`@Before`): 在目标方法执行之前执行。
- 后置通知(`@After`): 在目标方法执行之后执行,无论目标方法是否抛出异常。
- 返回通知(`@AfterReturning`): 在目标方法正常返回结果后执行。
- 异常通知(`@AfterThrowing`): 在目标方法抛出异常时执行。
- 环绕通知(`@Around`): 环绕目标方法的执行,可以在目标方法执行前后进行额外的操作,并且可以控制目标方法是否执行。

42. 如何在 AOP 中传递参数？

- 答：在通知方法中可以通过参数来接收目标方法的参数。对于前置通知、后置通知、返回通知和异常通知，可以在方法参数中使用`JoinPoint`对象来获取目标方法的参数信息。对于环绕通知，方法参数中需要使用`ProceedingJoinPoint`对象，通过调用`proceed`方法来执行目标方法，并可以在`proceed`方法中传递参数。

43. Spring AOP 和 AspectJ 的关系是什么？

- 答：Spring AOP 是 Spring 框架提供的面向切面编程的实现，它基于动态代理，主要用于处理 Spring 容器中的 Bean。AspectJ 是一个功能更强大的 AOP 框架，它支持编译时织入、加载时织入和运行时织入等多种织入方式。Spring AOP 在某些情况下可以集成 AspectJ，使用 AspectJ 的注解和语法来定义切面和通知，提高 AOP 的功能和灵活性。

44. 如何在 AOP 中使用切入点表达式？

- 答：切入点表达式用于定义哪些连接点会被织入切面的通知。Spring AOP 支持多种切入点表达式，如`execution`、`within`、`@annotation`等。例如，`execution( com.example.service.\*(..))`表示匹配`com.example.service`包下所有类的所有方法。可以在`@Pointcut`注解中定义切入点表达式，然后在通知注解中引用该切入点。

45. Spring AOP 的性能开销如何？

- 答：Spring AOP 的性能开销主要取决于代理的创建和方法调用时的额外处理。使用 JDK 动态代理和 CGLIB 代理创建代理对象会有一定的性能开销，但这个开销通常是一次性的。在方法调用时，由于需要执行通知逻辑，会比直接调用目标方法多一些处理步骤，不过对于大多数应用来说，这种性能开销是可以接受的。可以通过合理设计切面和切入点，减少不必要的代理和通知执行，来降低性能开销。

Spring MVC

46. Spring MVC 中如何处理异常？

- 答：可以通过以下几种方式处理异常：
- 实现 `HandlerExceptionResolver` 接口，自定义异常处理器，在 `resolveException` 方法中处理异常并返回相应的 `ModelAndView`。
- 使用 `@ExceptionHandler` 注解在控制器类中定义异常处理方法，该方法可以处理该控制器类中抛出的指定类型的异常。
- 使用 `@ControllerAdvice` 注解定义全局异常处理器，结合 `@ExceptionHandler` 注解可以处理所有控制器中抛出的异常。

47. Spring MVC 中如何进行数据验证？

- 答：可以使用 JSR 303(Bean Validation)规范进行数据验证。在 Java 对象的属性上使用`@NotNull`、`@Size`等注解进行验证规则的定义。在控制器方法的参数上使用`@Valid`注解触发验证，同时可以在方法参数中添加`BindingResult`对象来获取验证结果。如果验证失败，可以根据`BindingResult`中的错误信息进行相应的处理。

48. Spring MVC 中如何实现 RESTful API ?

- 答：在 Spring MVC 中可以使用`@RestController`注解标记控制器类，该注解相当于`@Controller`和`@ResponseBody`的组合，使得控制器方法的返回值会直接作为响应体返回，而不会进行视图解析。使用`@RequestMapping`或其派生注解（如`@GetMapping`、`@PostMapping`等）来定义不同 HTTP 方法的请求映射。同时，可以使用`@PathVariable`、`@RequestParam`等注解来获取请求参数。

49. Spring MVC 中如何进行跨域请求处理？

- 答：可以通过以下几种方式处理跨域请求：
- 在控制器方法上使用`@CrossOrigin`注解，该注解可以指定允许跨域的源、请求方法等信息。
- 实现`CorsFilter`过滤器，在过滤器中设置响应头，允许跨域请求。

- 在 Spring Boot 项目中，可以通过配置`CorsRegistry`来全局配置跨域请求。

50. Spring MVC 中视图解析器的作用是什么？

- 答：视图解析器的作用是将控制器方法返回的视图名称解析为具体的视图对象。Spring MVC 提供了多种视图解析器，如`InternalResourceViewResolver`用于解析 JSP 视图，`ThymeleafViewResolver`用于解析 Thymeleaf 模板视图等。视图解析器会根据配置的规则，将视图名称转换为实际的视图资源路径，并将其返回给`DispatcherServlet`进行视图渲染。

Spring 数据访问

51. Spring JDBC 和传统 JDBC 相比有哪些优势？

- 答：Spring JDBC 相比传统 JDBC 有以下优势：
- 简化了 JDBC 操作，减少了大量的样板代码，如`Connection`、`Statement`和`ResultSet`的创建、关闭等操作。
- 提供了统一的 API，使得数据库操作更加方便和规范。
- 支持事务管理，通过`JdbcTemplate`可以方便地进行事务控制。
- 提供了异常处理机制，将 JDBC 的`SQLException`转换为 Spring 的`DataAccessException`，更易于处理和理解。

52. Spring Data JPA 中如何自定义查询方法？

- 答：可以在继承`JpaRepository`等接口的自定义接口中定义方法名，Spring Data JPA 会根据方法名自动解析查询逻辑。例如，方法名`findByUsername`会根据`username`属性进行查询。也可以使用`@Query`注解在方法上定义自定义的 SQL 或 JPQL 查询语句，同时可以使用`@Param`注解来指定查询参数。

53. 如何在 Spring 中使用事务模板进行编程式事务管理？

- 答：首先需要配置`PlatformTransactionManager`，如`DataSourceTransactionManager`。然后创建`TransactionTemplate`对象，并将`PlatformTransactionManager`注入到`TransactionTemplate`中。在需要进行事务管理的代码中，调用`TransactionTemplate`的`execute`方法，在`execute`方法的回调函数中编写事务操作的代码。例如：

```
1 TransactionTemplate transactionTemplate = new
  TransactionTemplate(transactionManager);
2 transactionTemplate.execute(status -> {
3     // 事务操作代码
4     return null;
5 });
```

54. Spring Data JPA 中如何进行分页和排序查询？

- 答：可以在自定义的Repository接口中定义方法，方法参数中使用

`Pageable` 对象来实现分页和排序查询。`Pageable` 对象可以通过
`PageRequest.of` 方法创建，指定页码、每页记录数和排序规则。例如：

```
1 Page<User> findAll(Pageable pageable);
```

在调用该方法时，传入`Pageable`对象即可实现分页和排序查询。

55. 如何在 Spring 中配置多数据源？

- 答：可以通过以下步骤配置多数据源：
- 定义多个数据源 Bean，分别配置不同的数据库连接信息。
- 配置多个`JdbcTemplate`或`EntityManagerFactory`，每个对应一个数据源。
- 根据需要在不同的服务类中注入不同的`JdbcTemplate`或`EntityManagerFactory`来操作不同的数据源。

Spring Boot

56. Spring Boot 的自动配置原理是什么？

- 答：Spring Boot 的自动配置基于`@EnableAutoConfiguration`注解。当应用启动时，Spring Boot 会扫描`META-INF/spring.factories`文件，该文件中定义了一系列自动配置类。Spring Boot 会根据类路径下的依赖和配置属性，自动判断是否需要启用这些自动配置类。如果满足条件，就会将这些自动配置类

加载到 Spring 容器中，从而实现自动配置的功能。

57. 如何在 Spring Boot 中自定义自动配置？

- 答：可以创建一个配置类，使用`@Configuration`注解标记。在配置类中使用`@ConditionalOnClass`、`@ConditionalOnMissingBean`等条件注解来控制自动配置的生效条件。然后将该配置类添加到`META-INF/spring.factories`文件中，指定`org.springframework.boot.autoconfigure.EnableAutoConfiguration`为键，配置类的全限定名作为值。

58. Spring Boot 中如何配置外部属性文件？

- 答：可以在`application.properties`或`application.yml`文件中配置属性。也可以通过`@PropertySource`注解指定额外的属性文件。例如：

```
1  @Configuration
2  @PropertySource("classpath:custom.properties")
3  public class AppConfig {
4      // 配置类内容
5  }
```

在`custom.properties`文件中定义属性，然后在代码中可以通过`@Value`注解或`Environment`对象来获取属性值。

59. Spring Boot Actuator 的作用是什么？

- 答：Spring Boot Actuator 提供了一系列的端点(Endpoint)，用于监控和管理 Spring Boot 应用。通过这些端点，可以获取应用的运行状态、健康信息、内存使用情况、线程信息等。可以通过 HTTP 请求或 JMX 等方式访问这些端点，方便对应用进行监控和调试。

60. 如何在 Spring Boot 中集成 Swagger？

- 答：首先在项目中添加Swagger的依赖，如`springfox - swagger2`和`springfox - swagger - ui`。然后创建一个配置类，使用`@Configuration`和`@EnableSwagger2`注解标记，在配置类中定义`Docket`对象，配置Swagger的相关信息，如API的分组、扫描的包等。最后启动应用，访问`/swagger - ui.html`即可查看 Swagger UI 界面，对 API 进行文档化和测试。

Spring Cloud

61. Spring Cloud 中常用的组件有哪些？

- 答：常用的组件包括：
- Eureka：服务注册与发现组件，用于实现微服务的注册和发现。
- Ribbon：客户端负载均衡组件，为服务调用提供负载均衡功能。
- Feign：声明式的 HTTP 客户端，简化服务之间的调用。

- Hystrix：熔断器组件，用于实现服务的熔断和降级，防止服务雪崩。
- Zuul：API 网关组件，提供路由、过滤等功能。
- Config Server：配置中心组件，用于集中管理微服务的配置信息。

62. Spring Cloud Eureka 的工作原理是什么？

- 答：Eureka 包含 Eureka Server 和 Eureka Client。Eureka Server 是服务注册中心，负责接收和管理服务的注册信息。Eureka Client 分为服务提供者和服务消费者，服务提供者在启动时会向 Eureka Server 注册自己的信息，包括服务名称、IP 地址、端口等。服务消费者在启动时会从 Eureka Server 获取服务注册信息，并缓存到本地。当服务消费者需要调用服务时，会从本地缓存中获取服务提供者的信息，并进行负载均衡选择合适的服务实例进行调用。同时，服务提供者会定期向 Eureka Server 发送心跳信息，以表明自己的存活状态。

63. Spring Cloud Config 的配置刷新机制是怎样的？

- 答：在 Spring Cloud Config 中，当配置文件发生变化时，可以通过以下方式刷新配置：
- 手动刷新：使用`@RefreshScope`注解标记需要刷新配置的 Bean，然后通过调用`/actuator/refresh`端点来触发配置刷新。
- 自动刷新：可以结合 Spring Cloud Bus 和消息队列（如 RabbitMQ）实现自

动刷新。当配置文件发生变化时，Config Server 会发送消息到消息队列，所有监听该消息队列的服务实例会接收到消息并自动刷新配置。

64. Spring Cloud Hystrix 的熔断和降级机制是怎样的？

- 答：熔断机制：当服务调用失败的次数达到一定阈值时，Hystrix 会触发熔断，不再调用实际的服务，而是直接返回一个默认的响应。这样可以防止服务调用者长时间等待或出现大量的错误，避免服务雪崩。
- 降级机制：当服务不可用或出现异常时，Hystrix 会调用降级方法，返回一个预先定义的默认结果。降级方法可以是简单的静态数据，也可以是从缓存中获取的数据等，以保证服务的基本可用性。

65. Spring Cloud Gateway 的作用是什么？

- 答：Spring Cloud Gateway 是 Spring Cloud 官方提供的 API 网关，它可以作为微服务架构中的统一入口，提供路由、过滤、限流、熔断等功能。通过路由规则，可以将客户端的请求转发到不同的微服务实例。过滤功能可以对请求和响应进行预处理和后处理，如添加请求头、修改响应内容等。限流和熔断功能可以保护后端微服务，防止过多的请求导致服务崩溃。

Spring 缓存

66. Spring 中如何使用缓存？

- 答：可以使用 Spring 的缓存抽象来使用缓存。首先在配置类中使用 `@EnableCaching` 注解开启缓存功能。然后在需要使用缓存的方法上使用 `@Cacheable`、`@CachePut`、`@CacheEvict` 等注解。`@Cacheable` 注解用于缓存方法的返回值，当再次调用该方法时，如果缓存中存在对应的结果，则直接从缓存中获取；`@CachePut` 注解用于更新缓存；`@CacheEvict` 注解用于清除缓存。

67. Spring 支持哪些缓存管理器？

- 答：Spring 支持多种缓存管理器，如 `SimpleCacheManager`、`ConcurrentMapCacheManager`、`EhCacheCacheManager`、`RedisCacheManager` 等。`SimpleCacheManager` 和 `ConcurrentMapCacheManager` 使用内存中的 `Map` 来存储缓存数据；`EhCacheCacheManager` 使用 EhCache 作为缓存实现；`RedisCacheManager` 使用 Redis 作为缓存存储。

68. 如何在 Spring 中配置 Redis 缓存？

- 答：首先在项目中添加 Redis 的依赖，如 `spring - boot - starter - data - redis`。然后在 `application.properties` 或 `application.yml` 文件中配置 Redis 的连接信息，如 `spring.redis.host`、`spring.redis.port` 等。接着在配置类中使用 `@Configuration` 注解标记，创建 `RedisCacheManager` 实例，并将其注入

到 Spring 容器中。最后在需要使用缓存的方法上使用缓存注解即可。

69. Spring 缓存注解的执行顺序是怎样的？

- 答：`@Cacheable` 注解的执行顺序是先检查缓存中是否存在对应的结果，如果存在则直接返回，否则执行方法并将结果存入缓存。`@CachePut` 注解会先执行方法，然后将方法的返回值存入缓存。`@CacheEvict` 注解会在方法执行前或执行后清除缓存，具体取决于 `beforeInvocation` 属性的值，默认是在方法执行后清除缓存。

70. 如何自定义 Spring 缓存注解的缓存键生成策略？

- 答：可以通过实现 `KeyGenerator` 接口来自定义缓存键生成策略。在实现类中重写 `generate` 方法，根据方法的参数和返回值等信息生成自定义的缓存键。然后在配置类中创建 `KeyGenerator` 的 Bean 实例，并在缓存注解中使用 `keyGenerator` 属性指定自定义的缓存键生成器。

Spring 安全性

71. Spring Security 的作用是什么？

- 答：Spring Security 是 Spring 框架提供的用于实现安全认证和授权的模块。它可以对 Web 应用程序进行全面的安全保护，包括用户认证（验证用户身份）、授权（控制用户对资源的访问权限）、防止跨站请求伪造（CSRF）、防止会话

固定攻击等。

72. 如何在 Spring Boot 中集成 Spring Security ?

- 答：在Spring Boot项目中添加`spring - boot - starter - security`依赖。

Spring Boot 会自动配置基本的安全机制，默认会要求所有请求都进行身份验证。可以通过创建一个配置类，继承`WebSecurityConfigurerAdapter`类，重写`configure`方法来进行自定义的安全配置，如配置登录页面、允许匿名访问的路径、用户认证和授权规则等。

73. Spring Security 中如何进行用户认证？

- 答：可以通过以下几种方式进行用户认证：
- 内存认证：在配置类中使用`inMemoryAuthentication`方法配置用户信息，将用户信息存储在内存中。
- JDBC 认证：使用`jdbcAuthentication`方法配置基于数据库的用户认证，从数据库中查询用户信息进行认证。
- LDAP 认证：使用`ldapAuthentication`方法配置基于 LDAP 的用户认证，从 LDAP 服务器中查询用户信息进行认证。
- 自定义认证：实现`UserDetailsService`接口，自定义用户信息的加载逻辑，然后在配置类中注入该实现类。

74. Spring Security 中如何进行授权管理？

- 答：可以在配置类的`configure`方法中使用`authorizeRequests`方法来配置授权规则。例如，使用`antMatchers`方法指定请求路径，使用`hasRole`、`hasAuthority`等方法指定用户角色或权限。只有具有相应角色或权限的用户才能访问指定的路径。也可以在控制器方法上使用`@PreAuthorize`、`@PostAuthorize`等注解进行细粒度的授权控制。

75. Spring Security 中如何防止 CSRF 攻击？

- 答：Spring Security 默认开启了 CSRF 保护。对于基于表单的 POST、PUT、DELETE 等请求，需要在表单中添加 CSRF 令牌。可以在 JSP 页面中使用`<input type="hidden" name="\$ { \_csrf. parameterName} " value="\$ {\_csrf.token}"/>`来添加 CSRF 令牌。对于 RESTful API，可以通过配置`CsrfFilter`来禁用 CSRF 保护，或者使用`CsrfTokenRepository`来管理 CSRF 令牌。

Spring WebFlux

76. 什么是 Spring WebFlux？

- 答：Spring WebFlux 是 Spring 框架在 5.0 版本引入的响应式 Web 框架，它基于 Reactor 项目实现了响应式编程模型。与传统的 Spring MVC 不同，Spring WebFlux 支持非阻塞 I/O 和异步处理，能够处理大量的并发请求，提

高应用的性能和可扩展性。

77. Spring WebFlux 和 Spring MVC 的区别是什么？

- 答：主要区别包括：
- 编程模型：Spring MVC 基于 Servlet API，是同步阻塞的编程模型；Spring WebFlux 基于响应式编程模型，支持非阻塞 I/O 和异步处理。
- 性能：Spring WebFlux 在处理大量并发请求时性能更好，因为它不需要为每个请求创建一个线程，减少了线程上下文切换的开销。
- 适用场景：Spring MVC 适用于传统的 Web 应用开发；Spring WebFlux 适用于对性能和可扩展性要求较高的场景，如实时数据处理、高并发的 API 服务等。

78. 如何在 Spring WebFlux 中创建 RESTful API？

- 答：可以使用 `@RestController` 注解标记控制器类，使用 `@GetMapping`、`@PostMapping` 等注解定义不同 HTTP 方法的请求映射。方法的返回值可以是 `Mono` 或 `Flux` 类型，分别表示单个响应结果或多个响应结果。例如：

```
1  @RestController
2  public class UserController {
3      @GetMapping("/users")
4      public Flux<User> getUsers() {
5          // 返回 Flux<User>类型的响应
6          return userService.getUsers();
7      }
8  }
```

79. Spring WebFlux 中如何进行数据验证？

- 答：与 Spring MVC 类似，可以使用 JSR 303 (Bean Validation) 规范进行数据验证。在Java对象的属性上使用验证注解，在控制器方法的参数上使用`@Valid`注解触发验证。不同的是，在 Spring WebFlux 中，验证结果可以通过`BindingResult`对象或`Mono<BindingResult>`对象来获取。

80. Spring WebFlux 中如何处理异常？

- 答：可以通过实现`WebExceptionHandler`接口自定义异常处理器，在`handle`方法中处理异常并返回响应。也可以在控制器类中使用`@ExceptionHandler`注解定义异常处理方法，处理该控制器类中抛出的异常。

Spring 事务管理

81. Spring 事务的传播行为有哪些？

- 答：Spring 事务的传播行为定义了多个事务方法相互调用时，事务如何传

播。主要的传播行为包括：

- ``PROPAGATION_REQUIRED``：如果当前存在事务，则加入该事务；如果不存在事务，则创建一个新的事务。
- ``PROPAGATION_SUPPORTS``：如果当前存在事务，则加入该事务；如果不存在事务，则以非事务方式执行。
- ``PROPAGATION_MANDATORY``：如果当前存在事务，则加入该事务；如果不存在事务，则抛出异常。
- ``PROPAGATION_REQUIRES_NEW``：无论当前是否存在事务，都会创建一个新的事务，并挂起当前事务（如果存在）。
- ``PROPAGATION_NOT_SUPPORTED``：以非事务方式执行，如果当前存在事务，则挂起该事务。
- ``PROPAGATION_NEVER``：以非事务方式执行，如果当前存在事务，则抛出异常。
- ``PROPAGATION_NESTED``：如果当前存在事务，则在嵌套事务中执行；如果不存在事务，则创建一个新的事务。

82. 如何在 Spring 中配置分布式事务？

- 答：可以使用 Atomikos 等分布式事务管理器来配置分布式事务。首先在项

目中添加 Atomikos 的依赖，然后配置数据源和事务管理器，将数据源包装成 Atomikos 数据源，将事务管理器配置为 Atomikos 事务管理器。在需要进行分布式事务管理的方法上使用`@Transactional`注解，指定事务管理器。

83. Spring 事务的隔离级别有哪些？

- 答：Spring 事务的隔离级别与数据库的隔离级别相对应，主要包括：
- `ISOLATION\_DEFAULT`：使用数据库的默认隔离级别。
- `ISOLATION\_READ\_UNCOMMITTED`：允许读取未提交的数据，可能会出现脏读、不可重复读和幻读问题。
- `ISOLATION\_READ\_COMMITTED`：只允许读取已提交的数据，避免了脏读，但可能会出现不可重复读和幻读问题。
- `ISOLATION\_REPEATABLE\_READ`：保证在同一个事务中多次读取同一数据的结果是一致的，避免了脏读和不可重复读，但可能会出现幻读问题。
- `ISOLATION\_SERIALIZABLE`：最高的隔离级别，保证事务串行执行，避免了脏读、不可重复读和幻读问题，但会影响并发性能。

84. 如何在 Spring 中手动管理事务？

- 答：可以通过`PlatformTransactionManager`手动管理事务。首先获取`PlatformTransactionManager`实例，然后创建`TransactionDefinition`对象，

设置事务的传播行为、隔离级别等属性。接着调用`PlatformTransactionManager`的`getTransaction`方法开启事务，返回一个`TransactionStatus`对象。在事务操作完成后，根据操作结果调用`PlatformTransactionManager`的`commit`方法提交事务或`rollback`方法回滚事务。例如：

```
1 PlatformTransactionManager transactionManager = ...;
2 TransactionDefinition def = new DefaultTransactionDefinition();
3 TransactionStatus status = transactionManager.getTransaction(def);
4 try {
5     // 事务操作代码
6     transactionManager.commit(status);
7 } catch (Exception e) {
8     transactionManager.rollback(status);
9 }
```

85. Spring 事务管理中，@Transactional 注解的参数有哪些作用？

- 答：`@Transactional`注解的常用参数及作用如下：
- `propagation`：指定事务的传播行为，默认为`PROPAGATION\_REQUIRED`。
- `isolation`：指定事务的隔离级别，默认为`ISOLATION\_DEFAULT`。
- `readOnly`：指定事务是否为只读事务，默认为`false`。

- `timeout`：指定事务的超时时间，单位为秒。
- `rollbackFor`：指定哪些异常会导致事务回滚。
- `noRollbackFor`：指定哪些异常不会导致事务回滚。

Spring 与其他框架集成

86. 如何在 Spring 中集成 MyBatis？

- 答：可以通过 `MyBatis - Spring` 框架来集成 MyBatis 和 Spring。首先在项目中添加 `mybatis - spring` 和 `mybatis` 的依赖。然后配置数据源和 `SqlSessionFactoryBean`，将数据源注入到 `SqlSessionFactoryBean` 中。接着创建 Mapper 接口和对应的 Mapper XML 文件，将 Mapper 接口注册到 Spring 容器中。可以使用 `@MapperScan` 注解扫描 Mapper 接口所在的包，或者使用 `MapperFactoryBean` 手动注册 Mapper 接口。

87. 如何在 Spring 中集成 Hibernate？

- 答：可以通过 Spring 的 `LocalSessionFactoryBean` 来集成 Hibernate。首先在项目中添加 `hibernate - core` 和 `spring - orm` 等依赖。然后配置数据源和 `LocalSessionFactoryBean`，在 `LocalSessionFactoryBean` 中设置 Hibernate 的配置属性和实体类所在的包。接着创建 `HibernateTransactionManager` 来管理事务。在需要使用 Hibernate 的服务

类中注入`SessionFactory`，通过`SessionFactory`获取`Session`对象进行数据库操作。

88. 如何在 Spring 中集成 RabbitMQ ?

- 答：在Spring Boot项目中添加`spring - boot - starter - amqp`依赖。然后在`application.properties`或`application.yml`文件中配置 RabbitMQ 的连接信息，如`spring.rabbitmq.host`、`spring.rabbitmq.port`等。创建配置类，使用`@Configuration`注解标记，在配置类中定义`ConnectionFactory`、`RabbitTemplate`和`SimpleRabbitListenerContainerFactory`等Bean。使用`@RabbitListener`注解在方法上监听 RabbitMQ 的队列，使用`RabbitTemplate`发送消息到队列。

89. 如何在 Spring 中集成 Redis ?

- 答：在 Spring Boot 项目中添加`spring - boot - starter - data - redis`依赖。在`application.properties`或`application.yml`文件中配置 Redis 的连接信息。创建配置类，使用`@Configuration`注解标记，在配置类中定义`RedisConnectionFactory`和`RedisTemplate`等 Bean。可以使用`RedisTemplate`或`StringRedisTemplate`来操作 Redis，如存储和获取数据、设置过期时间等。

90. 如何在 Spring 中集成 Elasticsearch ?

- 答 在 Spring Boot 项目中添加`spring - boot - starter - data - elasticsearch`依赖。在`application.properties`或`application.yml`文件中配置 Elasticsearch 的连接信息，如`spring. elasticsearch. rest. uris`。创建实体类，使用`@Document`、`@Field`等注解标记实体类和属性。创建 Repository 接口，继承`ElasticsearchRepository`接口，通过该接口进行数据的增删改查操作。

Spring 性能优化

91. 如何优化 Spring 应用的启动时间？

- 答：可以通过以下方式优化 Spring 应用的启动时间：
- 减少自动配置的范围，使用`@SpringBootApplication`注解的`exclude`属性排除不需要的自动配置类。
- 优化依赖，移除不必要的依赖，减少类路径扫描的范围。
- 使用延迟初始化，通过`@Lazy`注解将 Bean 的初始化延迟到第一次使用时。
- 配置缓存，如使用 EhCache 或 Redis 缓存，减少重复的数据库查询和计算。

92. 如何优化 Spring MVC 应用的性能？

- 答：可以从以下几个方面优化 Spring MVC 应用的性能：
- 缓存：使用 Spring 的缓存抽象或第三方缓存框架对频繁访问的数据进行缓存。

- 异步处理：使用`@Async`注解或`WebAsyncTask`等方式实现异步处理，提高应用的并发处理能力。
- 优化数据库查询：使用索引、优化 SQL 语句、减少不必要的数据库查询。
- 压缩响应数据：使用 Gzip 等压缩算法对响应数据进行压缩，减少数据传输量。
- 静态资源优化：将静态资源（如 CSS、JavaScript、图片等）进行合并、压缩和缓存。

93. 如何优化 Spring AOP 的性能？

- 答：可以通过以下方式优化 Spring AOP 的性能：
- 合理设计切入点，避免切入点表达式过于复杂，减少不必要的代理和通知执行。
- 避免在性能敏感的方法上使用 AOP，或者使用更轻量级的 AOP 实现。
- 对于频繁调用的方法，可以考虑将 AOP 逻辑内联到方法中，避免代理的开销。

94. 如何优化 Spring 数据访问的性能？

- 答：可以采取以下措施优化 Spring 数据访问的性能：
- 使用连接池：配置高性能的连接池，如 HikariCP、Druid 等，减少数据库连

接的创建和销毁开销。

- 批量操作：使用批量插入、更新和删除操作，减少数据库交互次数。
- 缓存查询结果：使用 Spring 的缓存抽象或第三方缓存框架对数据库查询结果进行缓存。
- 优化 SQL 语句：使用索引、避免全表扫描、合理使用连接和子查询等。
- 数据库调优：对数据库进行性能调优，如调整参数、优化表结构等。

95. 如何监控 Spring 应用的性能？

- 答：可以使用以下工具和方法监控 Spring 应用的性能：
- Spring Boot Actuator：提供了一系列的端点，如`/actuator/metrics`、`/actuator/health`等，用于获取应用的性能指标和健康信息。
- 性能监控工具：如 VisualVM、YourKit 等，用于监控应用的 CPU 使用情况、内存使用情况、线程信息等。
- APM 工具：如 Zipkin、SkyWalking 等，用于分布式系统的性能监控和链路追踪。

Spring 高级特性

96. Spring 中的 spel 表达式是什么？

- 答：SpEL (Spring Expression Language) 是 Spring 框架提供的一种表达式语言，用于在运行时计算表达式的值。它可以在配置文件、注解属性等地方使用，支持访问对象属性、调用方法、进行算术运算、逻辑运算等操作。例如，在`@Value`注解中可以使用SpEL表达式来获取配置属性的值，如`@Value("#{systemProperties['user.home']})"`。

97. Spring 中的 MBean 是什么？

- 答：MBean (Managed Bean) 是 Java Management Extensions (JMX) 中的一个概念，用于管理和监控 Java 对象。在 Spring 中，可以将 Spring Bean 注册为 MBean，通过 JMX 控制台对其进行管理和监控。可以使用`@ManagedResource`、`@ManagedAttribute`和`@ManagedOperation`等注解将 Spring Bean 标记